

OpenClaw Three Dialogue Configuration Tests

OpenClaw Three Dialogue Configuration Tests

1. Course Overview
2. Three Dialogue Methods Introduction
 - 2.1 WhatsApp Method
 - 2.2 TUI Method
 - 2.3 Voice Interaction Method
3. WhatsApp Configuration and Testing
 - 3.1 Install WhatsApp Plugin
 - 3.2 Testing Method
 - 3.3 Common Problems
4. TUI Dialogue Configuration and Testing
 - 4.1 Startup Method
 - 4.2 Testing Method
5. AI Voice Interaction Configuration and Testing
 - 5.1 Pre-Startup Preparation
 - 5.2 Startup Method
 - 5.3 Testing Method
 - 5.4 Judgment Criteria
 - 5.5 Common Problems
6. Comparison of Three Methods
7. Recommended Testing Order

1. Course Overview

This course will introduce three common dialogue methods for OpenClaw and provide their respective configuration and testing methods:

- WhatsApp dialogue
- Local TUI dialogue
- AI voice interaction dialogue

After completing this course, you will be able to choose the most suitable dialogue method for your scenario and complete basic verification.

2. Three Dialogue Methods Introduction

2.1 WhatsApp Method

Suitable scenarios:

- Mobile remote control
- Let multiple people experience OpenClaw together

Features:

- No need to stay by the device all the time
- Suitable for demonstrating "remote natural language control"

2.2 TUI Method

Suitable scenarios:

- Local debugging
- Quick testing
- Troubleshooting model and skill invocation issues

Features:

- Most direct
- Easiest to see commands and return results

2.3 Voice Interaction Method

Suitable scenarios:

- Complete AI interaction experience
- Voice wake-up and voice control
- Desktop assistant-style experience

Features:

- Strongest sense of interaction

3. WhatsApp Configuration and Testing

If you have the optional AI voice module + speaker, the TUI and WhatsApp conversations need to broadcast OpenClaw's replies. You can run the following program in the terminal. This is not needed for AI voice module conversations.

- Modify the TTS enable parameter

```
/home/pi/voice_to_openclaw/.env
```

```
ENABLE_TTS=true
TTS_ENGINE=xunfei_ws
TTS_API_URL=wss://tts-api.xfyun.cn/v2/tts
TTS_VOICE=x4_xiaoyan
TTS_SAMPLE_RATE=16000
TTS_VOLUME=50
TTS_TIMEOUT=60
XUNFEI_TTS_SPEED=50
XUNFEI_TTS_PITCH=50
XUNFEI_TTS_TEXT_ENCODING=UTF8
XUNFEI_TTS_SIGN_HOST=ws-api.xfyun.cn
WAKE_ACK_TEXT=I am here.
TTS_MAX_CHARS=200
TTS_TRUNCATE_MODE=smart
TTS_LOCK_PATH=/tmp/voice_to_openclaw_tts.lock
```

```
ENABLE_TTS=true # This parameter controls whether to broadcast replies. After
saving, run the program below in the terminal.
```

- Terminal input

```
cd ~/voice_to_openclaw
source .venv/bin/activate
python -m src.openclaw_session_tts
```

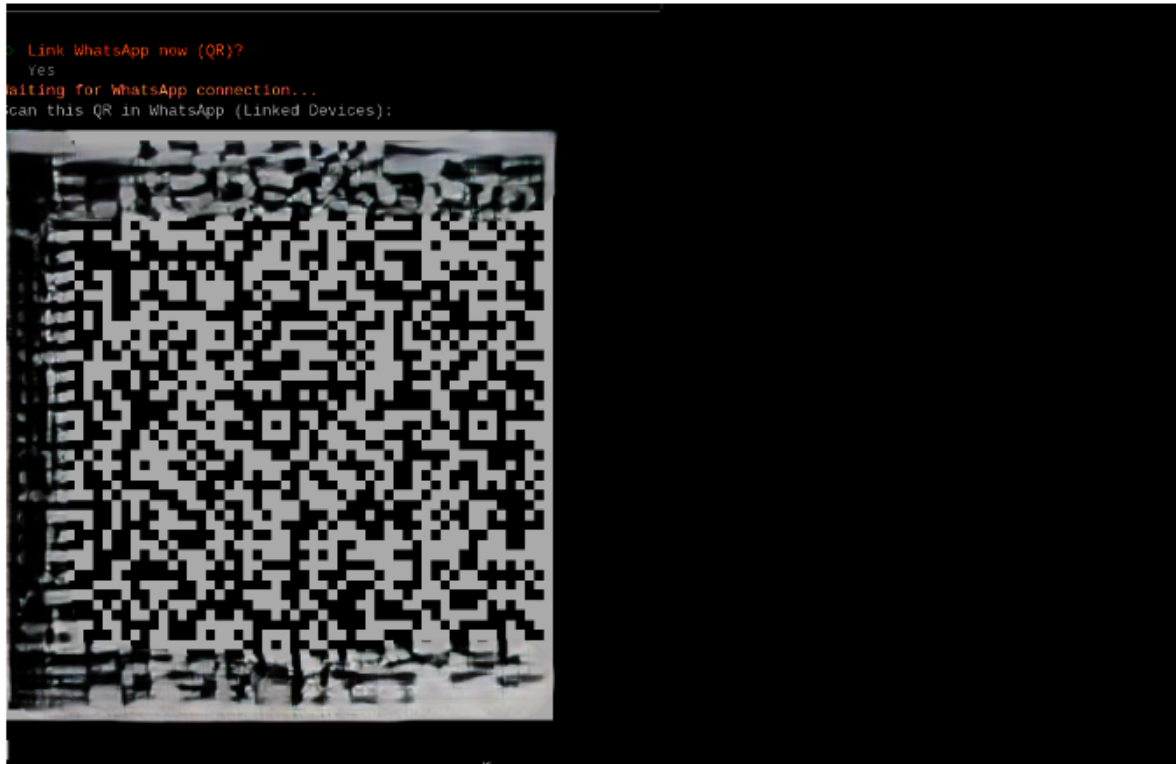
```
(.venv) pi@raspberrypi:~/voice_to_openclaw $ python -m src.openclaw_session_tts
[系统] OpenClaw 会话播报器已启动
[系统] 监听目录=/home/pi/.openclaw/agents/main/sessions
[系统] 监听渠道=feishu,tui,whatsapp
```

3.1 Install WhatsApp Plugin

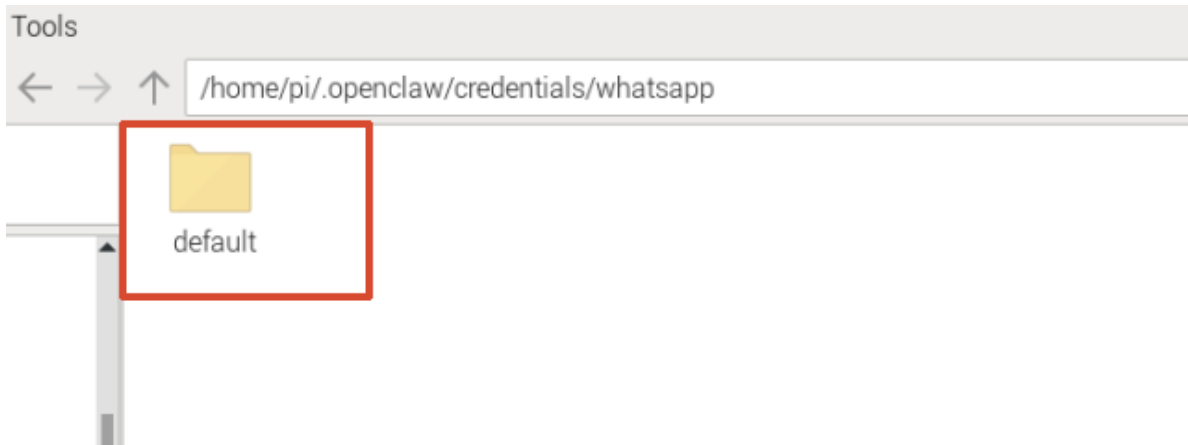
- Input in terminal:

```
openclaw channels login --channel whatsapp
```

- After execution, a QR code usually appears. (**Operate according to the [2.OpenClaw and WhatsApp Plugin Application] chapter in [12.OpenClaw Deployment and Basic Use]**)



Note: If no QR code is generated after running, you can delete the following files and then re-run the command.



- Scan the QR code on the mobile phone to connect. "Linked after restart" appears, proving the connection is successful

WhatsApp asked for a restart after pairing (code 515); creds are saved. Restarting connection once...
☒ Linked after restart; web session ready.

- Open the newly created conversation to see the conversation ID



- Fill this into "allowFrom" in the openclaw.json file:

```
nano /home/pi/.openclaw/openclaw.json
```

```

{} openclaw.json X
.openclaw > {} openclaw.json > {} channels > {} feishu > abc chat_id
144     "commands": {
145         "native": "auto",
146         "nativeSkills": "auto",
147         "restart": true,
148         "ownerDisplay": "raw"
149     },
150     "session": {
151         "dmScope": "per-channel-peer"
152     },
153     "channels": {
154         "whatsapp": {
155             "enabled": true,
156             "dmPolicy": "allowlist",
157             "selfChatMode": true,
158             "allowFrom": [
159                 "+8615338857526"
160             ],
161             "groupPolicy": "allowlist",
162             "debounceMs": 0,
163             "mediaMaxMb": 50
164         },

```

- Restart OpenClaw gateway

```
openclaw gateway restart
```

3.2 Testing Method

After creation, you can directly send test messages in WhatsApp, for example:

- Describe what you see

11:32



< 1



+86 153 3881 5798 (你)

给自己发消息

你想看哪张图片？它在什么路径？

11:14 ✓✓

昨天

Take a photo and describe what you see.

20:53 ✓✓

今天

Hello 11:30 ✓✓

Hello Master! 🖐️ I'm Xiaoya, your OpenClaw robot assistant.

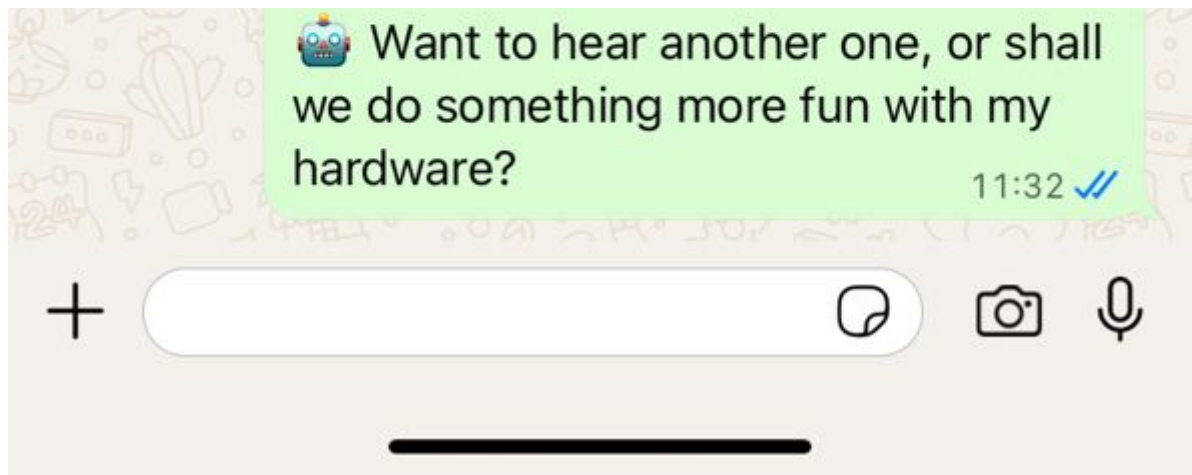
I'm awake and ready to help. Whether you need me to control hardware (servos, sensors, display, camera), run vision tasks, or work on skill development - just let me know what you'd like to do!

11:30 ✓✓

Tell me a joke. 11:32 ✓✓

Why don't robots ever panic?

Because they have excellent **circuits** under pressure! 😊



3.3 Common Problems

- If unable to create bot after scanning: check network first
- If created successfully but no reply: check if OpenClaw gateway restart is complete

4. TUI Dialogue Configuration and Testing

4.1 Startup Method

Input in terminal:

```
openclaw tui
```

If startup is successful, you will enter the local terminal dialogue interface.

```
pi@raspberrypi:~ $ openclaw tui
[!] OpenClaw 2026.3.12 (6472949)
  Your terminal just grew claws—type something and let the bot pinch the busywork.

openclaw tui - ws://127.0.0.1:18789 - agent main - session main
connecting | idle
agent main | session main | unknown | tokens ?

18:28:21 [plugins] feishu_chat: Registered feishu_chat, feishu_chat_members
18:28:21 [plugins] feishu_im: Registered feishu_im_user_message, feishu_im_user_fetch_resource, feishu_im_u
ser_get_messages, feishu_im_user_get_thread_messages, feishu_im_user_search_messages
18:28:21 [plugins] feishu_search: Registered feishu_search_doc_wiki
18:28:21 [plugins] feishu_drive: Registered feishu_drive_file, feishu_doc_comments, feishu_doc_media
18:28:21 [plugins] feishu_wiki: Registered feishu_wiki_space, feishu_wiki_space_node
18:28:21 [plugins] feishu_sheets: Registered feishu_sheet
18:28:21 [plugins] feishu_im: Registered feishu_im_bot_image
18:28:21 [plugins] Registered all OAPI tools (calendar, task, bitable, search, drive, wiki, sheets, im)
18:28:21 [plugins] feishu_doc: Registered feishu_fetch_doc, feishu_create_doc, feishu_update_doc
18:28:21 [plugins] feishu_oauth: Registered feishu_oauth tool

session agent:main:main
gateway connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens 0/1.0m (0%)
```

4.2 Testing Method

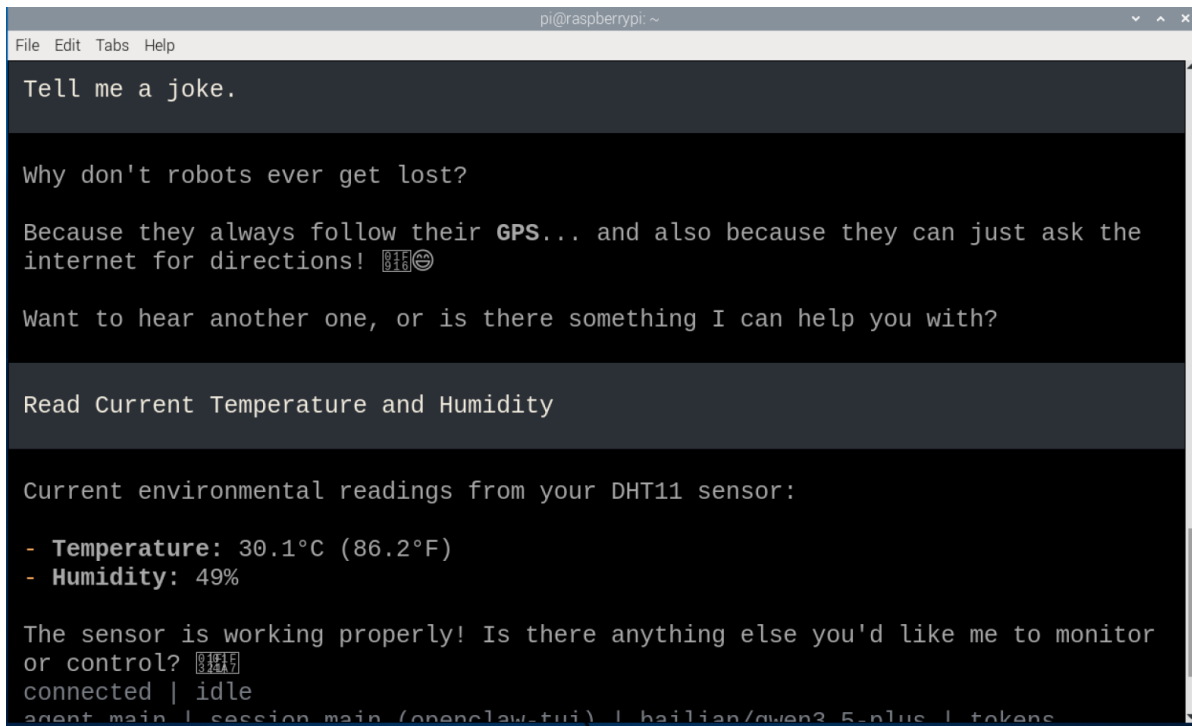
It is recommended to first input the simplest dialogue content:

- Tell me a joke
- Introduce yourself

If basic dialogue is normal, then test a hardware-related command, for example:

- Make RGB lights red

- Display "Hi" on OLED
- Read current temperature and humidity



```

pi@raspberrypi: ~
File Edit Tabs Help

Tell me a joke.

Why don't robots ever get lost?

Because they always follow their GPS... and also because they can just ask the
internet for directions! 🤖📶

Want to hear another one, or is there something I can help you with?

Read Current Temperature and Humidity

Current environmental readings from your DHT11 sensor:

- Temperature: 30.1°C (86.2°F)
- Humidity: 49%

The sensor is working properly! Is there anything else you'd like me to monitor
or control? 🤖📶
connected | idle
agent main | session main (openclaw-tui) | hailian/gwen2.5-plus | tokens

```

5. AI Voice Interaction Configuration and Testing

5.1 Pre-Startup Preparation

- Requires optional AI voice interaction module + speaker

5.2 Startup Method

```

cd /home/pi/voice_to_openclaw
source .venv/bin/activate
python -m src.main

```

5.3 Testing Method

If using serial wake mode, first trigger the wake "Hi,yahboom".

Then you can try saying:

- Tell me a joke
- Turn on RGB lights
- Display "Welcome to OpenClaw" on OLED

```
pi@raspberrypi: ~/voice_to_openclaw
File Edit Tabs Help

(.venv) pi@raspberrypi:~/voice_to_openclaw $ python -m src.main
2026-05-13 20:00:29 [INFO] voice_to_openclaw:run:26 - voice_to_openclaw starting
[系统] voice_to_openclaw 已启动
[系统] ASR=xunfei_ws/iat/en_us | 发布=gateway_chat | TTS=开/xunfei_ws | 唤醒=串口
[系统] 串口唤醒已启用: /dev/ttyUSB0@115200
[提示] 说“清空上下文”可以重置当前会话
[唤醒] 检测到串口信号, 开始新一轮对话
[听取] 请开始说话
[听取] 正在收音
[识别] 已捕获 2820 ms 语音, 正在转写

[你] Tell me a joke.
[发送] 正在交给 OpenClaw
[完成] OpenClaw 返回 200

[OpenClaw] Why did the robot go to therapy?

Because it had too many *bugs* in
its code! 🐛

...Don't worry, I'm working through my issues one debug
at a time. 😊
[信息] 回复已裁剪为播报文本: 149 -> 90 字
```

5.4 Judgment Criteria

If the voice method can complete recognition and receive OpenClaw reply, the following are probably normal:

- Microphone collection is normal
- ASR is normal
- OpenClaw publishing chain is normal
- [Optional] TTS is normal

5.5 Common Problems

- No response after wake: check serial wake configuration first
- Can wake but cannot recognize: check ASR and API-KEY
- Can recognize but not execute: check OpenClaw gateway configuration
- Has return but no broadcast: check TTS switch and speaker

6. Comparison of Three Methods

Method	Suitable Scenario	Advantages	Suggested Use
WhatsApp	Remote control, classroom demonstration	Convenient for mobile and multi-person experience	Demonstrate "remote natural language control"
TUI	Local debugging, command verification	Most direct, easiest to troubleshoot	First choice for basic debugging solution
Voice Interaction	Voice wake-up, complete experience	Strongest sense of interaction	Final demonstration solution

7. Recommended Testing Order

It is recommended to complete the three dialogue method tests in the following order:

1. First test `openclaw_tui`
2. Then test WhatsApp bot
3. Finally test AI voice interaction